

Autonomous Bug-Squashing Agent

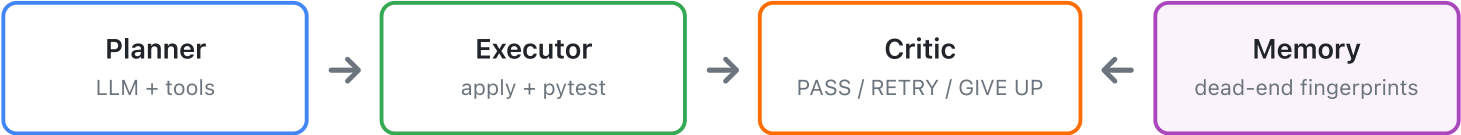
Planner → Executor → Critic loop for end-to-end Python bug repair

Abstract. LLM agent that locates and fixes Python bugs autonomously - read code, propose patch, run pytest, iterate until green. Evaluated on 52 synthetic cases and 3 injected defects in *pallets/click* v8.3.2 across 4 LLMs. Best result: **57% synth pass** (Qwen 2.5 72B) and **33% OSS pass** (Gemini 3 Pro), with **3.1–3.4× token reduction** and **2.5–2.7× latency improvement** over a Gemini CLI baseline at identical resolution outcomes.

Saransh Soni · Pranav Jitendra Trivedi · Yashashav Devalapalli Kamalraj

CMPE 258 - Spring 2026 | May 7, 2026

1. ARCHITECTURE - PLANNER → EXECUTOR → CRITIC



TOOLS AVAILABLE TO PLANNER

- `read_file(path, start, end)` - targeted reads, not full file
- `edit_file(path, line_range, new_block)` - atomic replace, AST-validated
- `run_bash(cmd)` - pytest oracle, sandboxed scope

KEY INVARIANTS

- **Per-case Memory** - fresh state, no cross-case leakage
- **Atomic patch** - write `.tmp` → `ast.parse()` → `os.replace()`
- **Path-traversal guard** - `realpath` scope check before write
- **Golden never fed** - used only for post-hoc evaluation

2. TWO COMPLEMENTARY BENCHMARKS

SYNTHETIC DATASET

Total cases	52
Valid runs	49 (3 skipped: pre-passing)
Difficulty tiers	Easy (40), Medium (9)
Source	Algorithmic mutations
Model evaluated	Qwen 2.5 72B (OpenRouter)
Budget	Max 10 retries, 180 s timeout

Controlled - isolates agent capability from repo complexity.

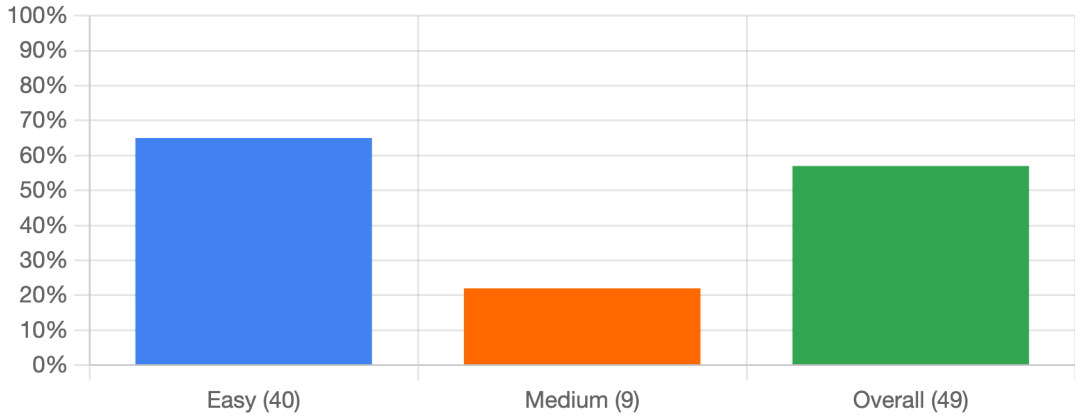
OSS REPOSITORY

Repository	pallets/click
Stars	~15,000 1,200+ source lines
Pinned commit	052c0060 (v8.3.2)
Injection	Deterministic string replace (idempotent)
Oracle	Repository's own pytest suite
Models evaluated	4 (Pro, Flash, Qwen, Gemma)

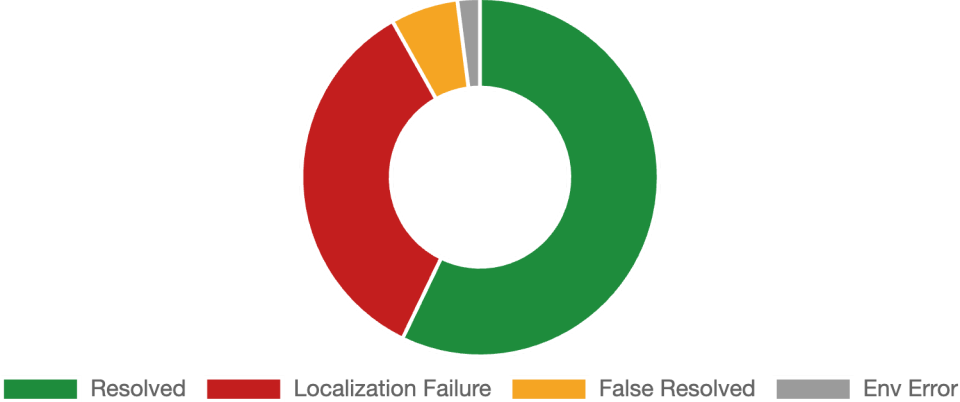
Real complexity - large files, multi-module dependencies.

3. SYNTHETIC DATASET RESULTS

PASS RATE BY DIFFICULTY



FAILURE MODE BREAKDOWN (49 CASES)



65%

Easy pass rate (26/40)

22%

Medium pass rate (2/9)

32 s

Average time per case

Localization failure (35%) dominates medium tier - model identifies symptom, patches wrong line.

4. OSS BUG CATALOG

Case	File	Injection	Tier
split_arg_upper	shell_completion.py	token → token.upper()	Easy
int_convert_off_by_one	types.py	value → value + 1	Medium
parser_prefixes_truncated	parser.py	{"-", "--"} → {"-"}	Hard

PREFLIGHT CONTRACT

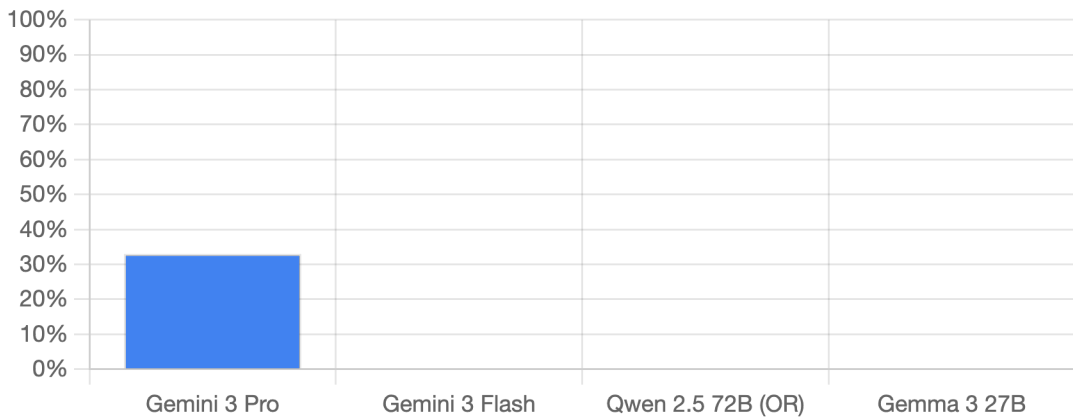
- Test must **FAIL** after injection
- Test must **PASS** after correct fix
- Pre-passing cases marked `invalid_benchmark_case`

WHY THESE THREE?

- **Easy** - single-token mutation in a small function
- **Medium** - type coercion bug, behavior visible only on specific inputs
- **Hard** - argument-parser semantics, multi-file ripple

5. OSS RESOLUTION MATRIX & FAILURE PROFILE

PASS RATE (% OF 3 CASES)

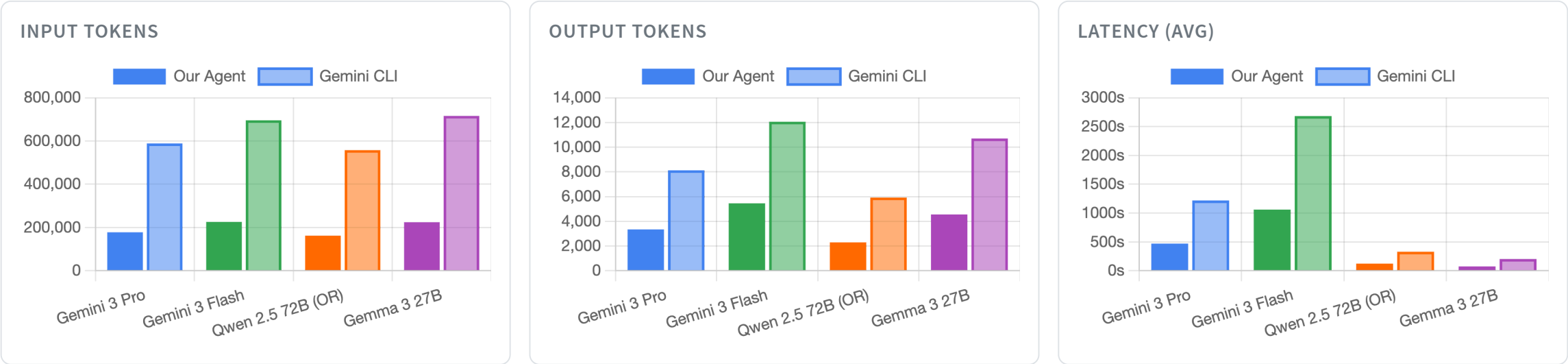


PER-CASE RESOLUTION

Model	Easy	Med	Hard	Total
Gemini 3 Pro	✓	✗	✗	1/3
Gemini 3 Flash	✗	✗	✗	0/3
Qwen 2.5 72B (OR)	✗	✗	✗	0/3
Gemma 3 27B	✗	✗	✗	0/3

Gemini 3 Pro resolves Easy in **2 retries / 119 s**. 87% of all OSS failures = pure **localization** (correct file, wrong line). Zero false-resolved, zero test regressions - patches syntactically safe.

6. EFFICIENCY: AGENT VS GEMINI CLI BASELINE



Model	Input cut	Output cut	Latency speedup
Gemini 3 Pro	3.31x	2.43x	2.59x
Gemini 3 Flash	3.08x	2.21x	2.53x
Qwen 2.5 72B (OR)	3.44x	2.58x	2.67x
Gemma 3 27B	3.19x	2.35x	2.72x

Identical resolution outcomes - efficiency gains are purely architectural.

7. WHAT DRIVES PERFORMANCE

MODEL REASONING → PASS RATE

Stronger reasoning models fix more bugs, independent of agent scaffolding. Gemini 3 Pro maps test traceback to mutation in 2 steps. Gemini 3 Flash, same scaffold, burns 10 retries on the same case.

On synth: Qwen 2.5 72B is 65% Easy → 22% Medium. Drop mirrors reasoning depth required, not architecture.

AGENT DESIGN → EFFICIENCY

- **Targeted tool calls** - read only relevant function, not full file
- **Dead-end memory** - skip patches already proven to fail
- **Structured retry context** - compact diff, not full traceback resend
- **Planner persona** - primes minimal JSON patch plans

Outcomes identical to baseline. Cost: 3.1–3.4× less input, 2.5–2.7× less time.

8. CONCLUSION & FUTURE WORK

Finding 1 - Model quality determines pass rate. Gap between models dwarfs any agent-level optimization. No retry logic compensates for insufficient reasoning depth on medium/hard.

Finding 2 - Agent design determines efficiency. 3.1–3.4× input-token reduction, 2.5–2.7× latency improvement, zero loss in resolution quality.

Finding 3 - Synthetic ≠ OSS difficulty. 57% synth vs ≤33% OSS reveals large-file localization and multi-module context as key unsolved challenges.

FUTURE DIRECTIONS

- **AST-guided patch targeting** - eliminate line-offset localization errors
- **Longer retry budgets** with curriculum difficulty scaling
- **Cross-file retrieval (RAG over repo)** for architectural bugs
- **Hard-tier synthetic cases** to bridge gap with OSS complexity

DEMO